

# Giới thiệu về NumPy

Mảng, vector hóa, broadcasting và tính toán số trong Python

Duc-Minh Vu

SLSCM Lab - FDA - NEU

August 3, 2026

# Lộ trình bài học

- 1 NumPy là gì?
- 2 Vì sao nên học NumPy?
- 3 Cài đặt và import
- 4 Tạo mảng NumPy
- 5 Thuộc tính của mảng
- 6 Indexing và slicing
- 7 Reshape, resize, stack, split
- 8 Broadcasting
- 9 Phép toán và aggregation
- 10 Universal functions
- 11 Boolean operations
- 12 Đại số tuyến tính
- 13 Số ngẫu nhiên và thống kê
- 14 Tích hợp và lỗi thường gặp

# Mục tiêu học tập

Sau bài học này, người học có thể:

- ▶ Giải thích vai trò của NumPy trong tính toán số.
- ▶ Phân biệt mảng NumPy với list thông thường của Python.
- ▶ Tạo mảng một chiều và nhiều chiều.
- ▶ Kiểm tra shape, size, ndim, và dtype.
- ▶ Sử dụng indexing, slicing, reshaping, stacking và splitting.
- ▶ Áp dụng phép toán vector hóa, broadcasting, aggregation và universal functions.
- ▶ Thực hiện các phép toán đại số tuyến tính và thống kê mô tả cơ bản.
- ▶ Sinh số ngẫu nhiên và hiểu vai trò của seed trong tái lập kết quả.
- ▶ Hiểu cách NumPy kết nối với Pandas, SciPy và Scikit-learn.

# Yêu cầu chuẩn bị và cài đặt

## Nền tảng nên có

- ▶ Biến, list, vòng lặp và hàm trong Python.
- ▶ Môi trường: Jupyter Notebook, JupyterLab, Google Colab hoặc Python IDE.

## Cài đặt và import

```
pip install numpy
```

```
import numpy as np  
print(np.__version__)
```

# NumPy là gì?

**NumPy**, viết tắt của **Numerical Python**, là thư viện Python cốt lõi cho tính toán số nhanh và hiệu quả.

Đối tượng trung tâm của NumPy là **mảng N chiều**, gọi là ndarray. Một mảng NumPy có thể biểu diễn:

- ▶ Vector một chiều.
- ▶ Ma trận hai chiều.
- ▶ Tensor ba chiều.
- ▶ Các cấu trúc số nhiều chiều hơn.

## Ý tưởng chính

NumPy lưu trữ dữ liệu đồng nhất hiệu quả và hỗ trợ phép toán nhanh trên toàn bộ mảng.

# NumPy cung cấp gì?

- ▶ Phép toán nhanh trên mảng.
- ▶ Tính toán vector hóa.
- ▶ Broadcasting.
- ▶ Hàm đại số tuyến tính.
- ▶ Hàm thống kê.
- ▶ Sinh số ngẫu nhiên.
- ▶ Tích hợp với Pandas, SciPy, Matplotlib và Scikit-learn.
- ▶ Nền tảng cho nhiều quy trình khoa học dữ liệu.

# Các đặc trưng chính

## ndarray

Cấu trúc dữ liệu trung tâm, lưu trữ phần tử cùng kiểu trong mảng nhiều chiều.

## Vector hóa và broadcasting

Vector hóa áp dụng phép tính lên toàn bộ mảng; broadcasting cho phép thao tác giữa các hình dạng tương thích.

## Đại số tuyến tính và thống kê

NumPy hỗ trợ nhân ma trận, định thức, nghịch đảo, trị riêng, trung bình, trung vị, phương sai và phân vị.

## Quick Check: NumPy là gì?

**Câu 1.** Cấu trúc dữ liệu chính trong NumPy là gì?

- ☐ A. DataFrame
- ☐ B. ndarray
- ☐ C. dictionary
- ☐ D. tuple

**Câu 2.** Đúng hay sai? NumPy được thiết kế chủ yếu cho tính toán số.

# Vì sao nên học NumPy?

NumPy là nền tảng hiệu quả cho tính toán số trong Python.

- ▶ Nhanh hơn nhiều vòng lặp Python trong nhiều tác vụ số nhờ vector hóa.
- ▶ Lưu trữ dữ liệu đồng nhất gọn hơn list Python.
- ▶ Cung cấp hàm tối ưu cho ma trận, đại số tuyến tính và thống kê.
- ▶ Cho phép biểu diễn công thức phức tạp bằng lệnh ngắn gọn.
- ▶ Là nền tảng cho Pandas, SciPy, Scikit-learn và nhiều thư viện dữ liệu khác.

# List Python và mảng NumPy

## List Python

```
values = [10, 2.5, "Python", True]
```

- ▶ Có thể chứa nhiều kiểu dữ liệu.
- ▶ Thao tác số thường cần vòng lặp.

## Mảng NumPy

```
import numpy as np
values = np.array([10, 20, 30, 40])
```

- ▶ Thường chứa một kiểu dữ liệu chung.
- ▶ Hỗ trợ phép toán số vector hóa.

## Ví dụ: nhân toàn bộ giá trị

### Dùng list Python

```
values = [1, 2, 3, 4]
result = []
for value in values:
    result.append(value * 10)
print(result)
```

### Dùng NumPy

```
import numpy as np
values = np.array([1, 2, 3, 4])
result = values * 10
print(result)
```

### Điểm cần nhớ

Trong NumPy, `values * 10` được áp dụng cho toàn bộ mảng mà không cần vòng lặp tường minh.

## Quick Check: Vì sao NumPy hiệu quả?

**Câu 1.** Vì sao mảng NumPy hiệu quả cho tính toán số?

- ☐ A. Vì luôn chứa văn bản
- ☐ B. Vì sử dụng cấu trúc đồng nhất và được tối ưu
- ☐ C. Vì không dùng bộ nhớ
- ☐ D. Vì tự động kết nối Internet

**Câu 2.** Biểu thức nào nhân mọi phần tử của mảng a với 10?

- ☐ A. `a * 10`
- ☐ B. `a.append(10)`
- ☐ C. `a.add("10")`
- ☐ D. `a.sort(10)`

# Tạo mảng NumPy từ list

```
import numpy as np  
  
a = [9, 3, 3, 5]  
arr = np.array(a)  
print(arr)
```

Kết quả

```
[9 3 3 5]
```

- ▶ `np.array()` chuyển list Python thành mảng NumPy.
- ▶ Mảng một chiều thường được dùng để biểu diễn vector.

# Mảng nhiều chiều

## Mảng hai chiều

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print(matrix)
```

## Mảng ba chiều

```
tensor = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])
print(tensor)
```

- ▶ Mảng hai chiều thường là ma trận.
- ▶ Mảng ba chiều có thể xem như tập nhiều ma trận hoặc tensor.

# Các hàm tạo mảng thông dụng

```
np.zeros(5)  
np.zeros((2, 3))  
np.ones((2, 3))  
np.full((2, 3), 7)
```

```
np.arange(0, 10, 2)  
np.linspace(0, 1, 5)  
np.eye(3)
```

## Ý nghĩa

- ▶ `arange`: tạo dãy theo bước nhảy.
- ▶ `linspace`: tạo số lượng điểm cách đều giữa hai đầu mút.
- ▶ `eye`: tạo ma trận đơn vị.

## Quick Check: Tạo mảng

**Câu 1.** Hàm nào chuyển list Python thành mảng NumPy?

- A. `np.array()`
- B. `np.list()`
- C. `np.convert()`
- D. `np.ndarray_list()`

**Câu 2.** Hàm nào tạo các giá trị cách đều giữa hai đầu mút?

- A. `np.linspace()`
- B. `np.stack()`
- C. `np.split()`
- D. `np.mean()`

# Thuộc tính của mảng

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print(arr.ndim)
print(arr.shape)
print(arr.size)
print(arr.dtype)
print(arr.itemsize)
```

| Thuộc tính | Ý nghĩa                    |
|------------|----------------------------|
| ndim       | Số chiều                   |
| shape      | Kích thước theo từng chiều |
| size       | Tổng số phần tử            |
| dtype      | Kiểu dữ liệu               |
| itemsize   | Số byte cho mỗi phần tử    |

## Quick Check: Thuộc tính mảng

**Câu 1.** Thuộc tính nào trả về hình dạng của mảng?

- ☐ A. shape
- ☐ B. mean
- ☐ C. append
- ☐ D. index

**Câu 2.** Thuộc tính nào trả về tổng số phần tử?

- ☐ A. size
- ☐ B. dtype
- ☐ C. ndim
- ☐ D. itemsize

# Indexing trong NumPy

## Mảng một chiều

```
arr = np.array([10, 20, 30, 40])  
print(arr[0])    # 10  
print(arr[2])    # 30  
print(arr[-1])   # 40
```

## Mảng hai chiều

```
matrix = np.array([  
    [1, 2, 3],  
    [4, 5, 6]  
)  
print(matrix[0, 1]) # 2  
print(matrix[1, 2]) # 6
```

# Slicing trong NumPy

```
arr = np.array([10, 20, 30, 40, 50])  
print(arr[1:4])    # [20 30 40]  
print(arr[::2])    # [10 30 50]
```

```
matrix = np.array([  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
)  
print(matrix[0:2, 1:3])
```

## Quy tắc

start:stop lấy từ start đến trước stop. Slicing hai chiều dùng cú pháp rows, columns.

# View và Copy

Nhiều phép slicing trả về **view**, tức là có thể dùng chung bộ nhớ với mảng gốc.

```
arr = np.array([10, 20, 30, 40])  
view = arr[1:3]  
view[0] = 999  
print(arr)
```

## Kết quả

```
[ 10 999 30 40]
```

Tạo bản sao độc lập bằng:

```
copy = arr[1:3].copy()
```

## Quick Check: Indexing và Slicing

**Câu 1.** `arr[0]` trả về gì?

- ☐ A. Phần tử đầu tiên
- ☐ B. Phần tử cuối cùng
- ☐ C. Hình dạng mảng
- ☐ D. Kích thước mảng

**Câu 2.** Đúng hay sai? Một lát cắt NumPy có thể dùng chung bộ nhớ với mảng gốc.

# Reshaping Arrays

```
arr = np.arange(12)
matrix = arr.reshape(3, 4)
print(matrix)
```

## Dùng -1

```
matrix = arr.reshape(2, -1)
```

NumPy tự suy ra chiều còn thiếu nếu tổng số phần tử phù hợp.

- ▶ `flatten()` thường trả về bản sao.
- ▶ `ravel()` thường trả về view nếu có thể.

# Resize, Stack và Split

## Resize

```
arr = np.array([1, 2, 3, 4])  
np.resize(arr, (2, 3))
```

Nếu kích thước mới lớn hơn, giá trị có thể được lặp lại.

## Stack

```
np.vstack((a, b))  
np.hstack((a, b))  
np.stack((a, b), axis=0)
```

## Split

```
arr = np.array([1,2,3,4,5,6])  
parts = np.split(arr, 3)
```

```
left, right = np.hsplit(matrix, 2)
```

## Quick Check: Reshape và Stack

**Câu 1.** Phương thức nào đổi hình dạng mảng?

- ☐ A. `reshape()`
- ☐ B. `mean()`
- ☐ C. `split()`
- ☐ D. `sort_index()`

**Câu 2.** Hàm nào ghép mảng theo chiều dọc?

- ☐ A. `np.vstack()`
- ☐ B. `np.mean()`
- ☐ C. `np.split()`
- ☐ D. `np.random()`

# Broadcasting là gì?

Broadcasting cho phép NumPy thực hiện phép toán giữa các mảng có hình dạng khác nhau nhưng tương thích.

## Broadcasting với số vô hướng

```
arr = np.array([1, 2, 3, 4])  
result = arr + 10  
print(result)
```

```
[11 12 13 14]
```

## Quy tắc cơ bản

So sánh từ chiều ngoài cùng bên phải. Hai chiều tương thích nếu chúng bằng nhau hoặc một trong hai bằng 1.

## Broadcasting giữa các mảng

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
row = np.array([10, 20, 30])
result = matrix + row
print(result)
```

### Kết quả

```
[[11 22 33] [14 25 36]]
```

### Lỗi hình dạng

Nếu hai mảng không tương thích, ví dụ (2,3) và (2,2), NumPy phát sinh lỗi broadcasting.

## Quick Check: Broadcasting

**Câu 1.** Broadcasting cho phép điều gì?

- ☐ A. Phép toán giữa các mảng có hình dạng khác nhau nhưng tương thích
- ☐ B. Truyền dữ liệu tự động qua Internet
- ☐ C. Chuyển mảng thành file
- ☐ D. Xóa mọi chiều của mảng

**Câu 2.** Hai chiều tương thích nếu bằng nhau hoặc:

- ☐ A. Một trong hai bằng 1
- ☐ B. Cả hai đều âm
- ☐ C. Cả hai là văn bản
- ☐ D. Tổng của chúng bằng 0

# Phép toán số học cơ bản

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
  
print(a + b)    # [5 7 9]  
print(a - b)    # [-3 -3 -3]  
print(a * b)    # [ 4 10 18]  
print(a / b)    # [0.25 0.4 0.5]  
print(a ** 2)   # [1 4 9]
```

## Phân biệt quan trọng

Toán tử `*` là nhân theo từng phần tử, không phải nhân ma trận.

# Aggregation Functions

```
arr = np.array([9, 3, 3, 5])  
print(arr.sum())  
print(arr.mean())  
print(arr.min())  
print(arr.max())  
print(np.median(arr))
```

```
matrix = np.array([  
    [1, 2, 3],  
    [4, 5, 6]  
)  
print(matrix.sum(axis=0))  
print(matrix.sum(axis=1))
```

- ▶ `axis=0`: tổng hợp theo hàng, trả kết quả theo cột.
- ▶ `axis=1`: tổng hợp theo cột, trả kết quả theo hàng.

## Quick Check: Phép toán và aggregation

**Câu 1.**  $a * b$  làm gì với hai mảng cùng hình dạng?

- A. Nhân theo từng phần tử
- B. Nghịch đảo ma trận
- C. Sắp xếp
- D. Ghép mảng

**Câu 2.** Hàm nào tính trung bình số học?

- A. `np.mean()`
- B. `np.stack()`
- C. `np.dot()`
- D. `np.reshape()`

# Universal Functions

**Universal function** hay **ufunc** thực hiện phép toán theo từng phần tử trên mảng.

```
arr = np.array([1, 4, 9, 16])  
print(np.sqrt(arr))  
print(np.exp(np.array([0, 1, 2])))
```

```
values = np.array([-3, -1, 2, 4])  
print(np.abs(values))  
print(np.round([1.234, 5.678], 2))
```

Ví dụ khác

`np.log()`, `np.sin()`, `np.cos()`, `np.tan()`.

# So sánh và lọc Boolean

```
arr = np.array([10, 20, 30, 40])  
print(arr > 20)  
selected = arr[arr > 20]  
print(selected)
```

```
selected = arr[(arr >= 20) & (arr <= 30)]  
print(selected)
```

- ▶ &: AND theo từng phần tử.
- ▶ |: OR theo từng phần tử.
- ▶ ~: NOT theo từng phần tử.

# Đại số tuyến tính trong NumPy

```
A = np.array([
    [1, 2],
    [3, 4]
])

print(np.dot(A, A))
print(A @ A)
print(A.T)
print(np.linalg.det(A))
print(np.linalg.inv(A))
```

## Ghi nhớ

$A * A$  là nhân theo từng phần tử;  $A @ A$  là nhân ma trận.

# Giải hệ tuyến tính và trị riêng

## Giải hệ $Ax = b$

```
A = np.array([[2, 1], [1, 3]])  
b = np.array([8, 13])  
x = np.linalg.solve(A, b)  
print(x)
```

## Trị riêng và vector riêng

```
eigenvalues, eigenvectors = np.linalg.eig(A  
)  
print(eigenvalues)  
print(eigenvectors)
```

- ▶ `np.inner()` và `np.outer()` cho tích trong và tích ngoài.
- ▶ `np.vdot()` xử lý liên hợp phức.

## Quick Check: Ufuncs và Linear Algebra

**Câu 1.** Hàm nào tính căn bậc hai theo từng phần tử?

- A. `np.sqrt()`
- B. `np.split()`
- C. `np.stack()`
- D. `np.size()`

**Câu 2.** Toán tử nào thực hiện nhân ma trận?

- A. `@`
- B. `%`
- C. `//`
- D. `&`

# Sinh số ngẫu nhiên

Trong mã mới, nên dùng bộ sinh số ngẫu nhiên hiện đại:

```
rng = np.random.default_rng(42)
```

```
rng.integers(1, 10, size=5)  
rng.uniform(0, 1, size=5)  
rng.normal(0, 1, size=5)
```

```
rng.binomial(n=10, p=0.5, size=5)  
rng.poisson(lam=3, size=5)  
rng.exponential(scale=2, size=5)
```

## Seed

Dùng cùng seed giúp tái lập chuỗi giả ngẫu nhiên trong môi trường tương thích.

# Các hàm thống kê

```
data = np.array([12, 15, 18, 20, 25])  
print(np.mean(data))  
print(np.median(data))  
print(np.var(data))  
print(np.std(data))  
print(np.percentile(data, 25))  
print(np.quantile(data, 0.75))
```

## Tổng thể và mẫu

Theo mặc định, `np.var()` và `np.std()` dùng `ddof=0`. Với quy ước mẫu chia cho  $N - 1$ , dùng `ddof=1`.

# Giá trị thiếu và NaN

NumPy thường biểu diễn giá trị số bị thiếu bằng `np.nan`.

```
data = np.array([10.0, 20.0, np.nan, 40.0])
print(np.mean(data))          # nan
print(np.nanmean(data))
print(np.nanmedian(data))
print(np.isnan(data))
clean_data = data[~np.isnan(data)]
```

## Lưu ý

NumPy hỗ trợ xử lý thiếu cơ bản. Quy trình dữ liệu bảng phức tạp thường dùng Pandas.

## Quick Check: Random và Statistics

**Câu 1.** Phương thức nào tạo bộ sinh số ngẫu nhiên hiện đại?

- ☐ A. `np.random.default_rng()`
- ☐ B. `np.random.file()`
- ☐ C. `np.create_random_array()`
- ☐ D. `np.random.text()`

**Câu 2.** Hàm nào tính trung bình nhưng bỏ qua NaN?

- ☐ A. `np.nanmean()`
- ☐ B. `np.mean_text()`
- ☐ C. `np.ignore()`
- ☐ D. `np.dropna()`

# Vectorization và hiệu năng

Vector hóa áp dụng một phép toán lên toàn bộ mảng cùng lúc.

```
a = np.arange(5)
result = a * 10
print(result)
```

## Ưu điểm

- ▶ Mã ngắn hơn và dễ đọc hơn.
- ▶ Nhanh hơn với mảng số lớn.
- ▶ Phù hợp với phân tích dữ liệu và tính toán khoa học.

```
labels = np.where(arr >= 30, "high", "low")
```

# Bộ nhớ và kiểu dữ liệu

```
arr = np.array([1, 2, 3, 4], dtype=np.int32)
print(arr.nbytes)
```

```
small_values = np.array([1, 2, 3, 4], dtype=np.int8)
print(small_values.dtype)
print(small_values.nbytes)
```

```
arr = np.array([1.2, 2.8, 3.5])
integers = arr.astype(np.int32)
print(integers)
```

## Lưu ý

Chọn kiểu dữ liệu nhỏ giúp tiết kiệm bộ nhớ nhưng phải đủ phạm vi và độ chính xác.

# Sắp xếp, tìm kiếm và giá trị duy nhất

```
arr = np.array([9, 3, 7, 1])  
sorted_arr = np.sort(arr)  
print(sorted_arr)
```

```
arr = np.array([10, 20, 30, 40])  
indices = np.where(arr > 20)  
print(indices)
```

```
arr = np.array([1, 2, 2, 3, 3, 3])  
values, counts = np.unique(arr, return_counts=True)
```

# Làm việc với ảnh

Ảnh số có thể được biểu diễn bằng mảng NumPy.

- ▶ Ảnh xám: mảng hai chiều.
- ▶ Ảnh màu: mảng ba chiều gồm chiều cao, chiều rộng và kênh màu.

```
image = np.array([
    [0, 128, 255],
    [255, 128, 0],
    [50, 100, 150]
])
print(image.shape)
brighter = np.clip(image + 30, 0, 255)
```

# Tích hợp với Pandas và Scikit-learn

## Pandas

```
arr = np.array([[1, 2], [3, 4]])  
df = pd.DataFrame(arr, columns=["A", "B"])  
array_from_df = df.to_numpy()  
df["A_sqrt"] = np.sqrt(df["A"])
```

## Scikit-learn

```
X = np.array([[1], [2], [3], [4]])  
y = np.array([2, 4, 6, 8])  
model = LinearRegression()  
model.fit(X, y)  
model.predict(np.array([[5]]))
```

## Quick Check: Hiệu năng và tích hợp

**Câu 1.** Ưu điểm chính của vector hóa là gì?

- A. Áp dụng phép toán lên mảng mà không cần vòng lặp Python tường minh
- B. Chuyển mọi mảng thành văn bản
- C. Loại bỏ nhu cầu bộ nhớ
- D. Ngăn chặn mọi lỗi

**Câu 2.** Trong học máy, mảng NumPy hai chiều thường biểu diễn:

- A. Ma trận đặc trưng
- B. Tên file
- C. Tiêu đề biểu đồ
- D. Trình cài đặt gói

# Các lỗi NumPy thường gặp

## Không khớp hình dạng

```
a = np.ones((2, 3))  
b = np.ones((2, 2))  
# a + b raises an error
```

## Reshape không hợp lệ

```
arr = np.arange(10)  
# arr.reshape(3, 4) raises an error
```

- ▶ Chia cho 0 có thể tạo `inf`.
- ▶ Kiểu số nguyên nhỏ có thể bị tràn.
- ▶ View có thể làm thay đổi mảng gốc ngoài ý muốn.

# Thực hành tốt khi dùng NumPy

- ▶ Import NumPy bằng `import numpy as np`.
- ▶ Ưu tiên vector hóa thay vì vòng lặp khi phù hợp.
- ▶ Kiểm tra `shape`, `ndim`, và `dtype` trước phép toán phức tạp.
- ▶ Chỉ dùng broadcasting khi hiểu rõ hành vi hình dạng.
- ▶ Phân biệt `*` và `@`.
- ▶ Dùng `.copy()` khi cần mảng độc lập.
- ▶ Chọn `dtype` phù hợp với phạm vi, độ chính xác và bộ nhớ.
- ▶ Dùng seed cố định khi cần tái lập kết quả.
- ▶ Xử lý rõ `NaN`, `inf` và tràn số.

# Tóm tắt nội dung

| Nội dung         | Ý chính                                         |
|------------------|-------------------------------------------------|
| NumPy            | Thư viện cốt lõi cho tính toán số               |
| ndarray          | Mảng đồng nhất N chiều                          |
| Vector hóa       | Phép toán mảng không cần vòng lặp tường minh    |
| Broadcasting     | Phép toán giữa các hình dạng tương thích        |
| Indexing/Slicing | Truy cập và trích xuất phần tử                  |
| Aggregation      | Tổng, trung bình, min, max                      |
| Linear algebra   | Nhân ma trận, nghịch đảo, định thức, trị riêng  |
| Statistics       | Trung bình, trung vị, phương sai, độ lệch chuẩn |
| Integration      | Kết nối Pandas, SciPy, Scikit-learn             |

# Bài tập thực hành: Tạo mảng và slicing

## Bài 1. Tạo mảng

- 1 Tạo mảng chứa các giá trị từ 1 đến 20.
- 2 Reshape thành ma trận  $4 \times 5$ .
- 3 In shape, ndim, size, và dtype.

## Bài 2. Indexing và slicing

Với ma trận  $4 \times 5$ , trích hàng đầu, cột cuối, khối trung tâm  $2 \times 3$ , và các giá trị chẵn bằng Boolean filtering.

# Bài tập thực hành: Statistics và broadcasting

## Bài 3. Thống kê

Tạo mảng 10 giá trị và tính mean, median, variance, sample standard deviation, min, max, và các phân vị 25%, 50%, 75%.

## Bài 4. Broadcasting

Tạo ma trận  $3 \times 4$ , tạo vector một chiều gồm 4 giá trị, cộng vector vào từng hàng và giải thích vì sao broadcasting hợp lệ.

# Bài tập thực hành: Linear Algebra và Missing Values

## Bài 5. Đại số tuyến tính

Cho  $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ ,  $b = [8, 13]$ . Tính định thức, nghịch đảo, giải  $Ax = b$ , và kiểm tra bằng  $A @ x$ .

## Bài 6. Dữ liệu thiếu

Với mảng có np.nan, đếm giá trị thiếu, tính mean bỏ qua NaN, loại bỏ NaN, và thay NaN bằng median.

# Phản tư cuối bài

- ❶ Khi nào nên dùng NumPy thay vì list Python?
- ❷ Vì sao shape rất quan trọng khi thao tác với mảng?
- ❸ Sự khác nhau giữa \* và @ là gì?
- ❹ Broadcasting giúp viết code ngắn hơn như thế nào?
- ❺ Vì sao cần cẩn thận với view, copy, NaN và dtype?